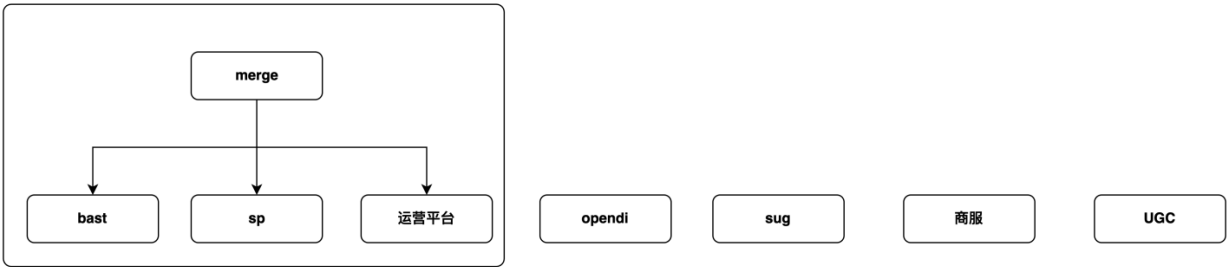
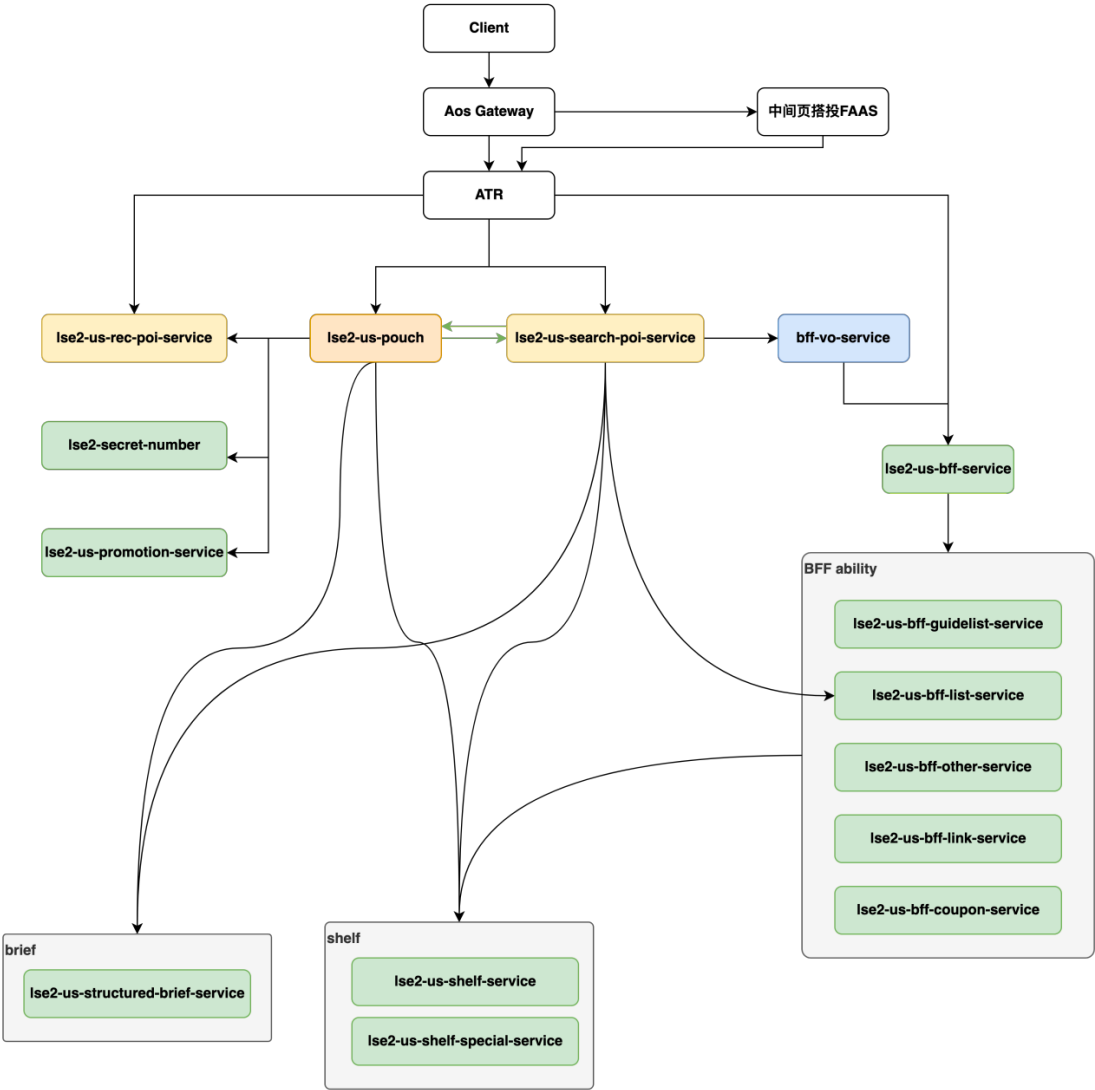


US架构思考

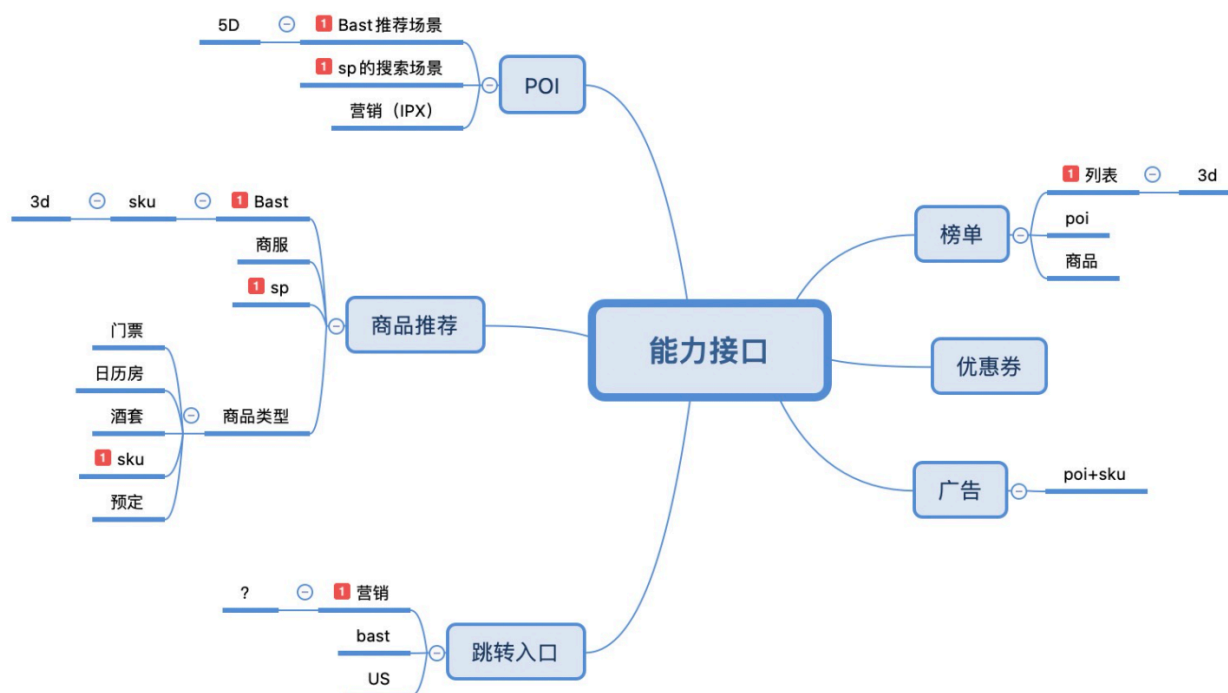
背景介绍

背景

US层现状调研 BFF能力建设



US服务基于MIF框架做了分层建设，将原有单体服务拆分为VO、Service、Ability三层多个服务；



同时基于原有调用链路，将能力归纳为跳转、列表、榜单、优惠、其他等服务，上层由bff-Service服务统一调度。

问题

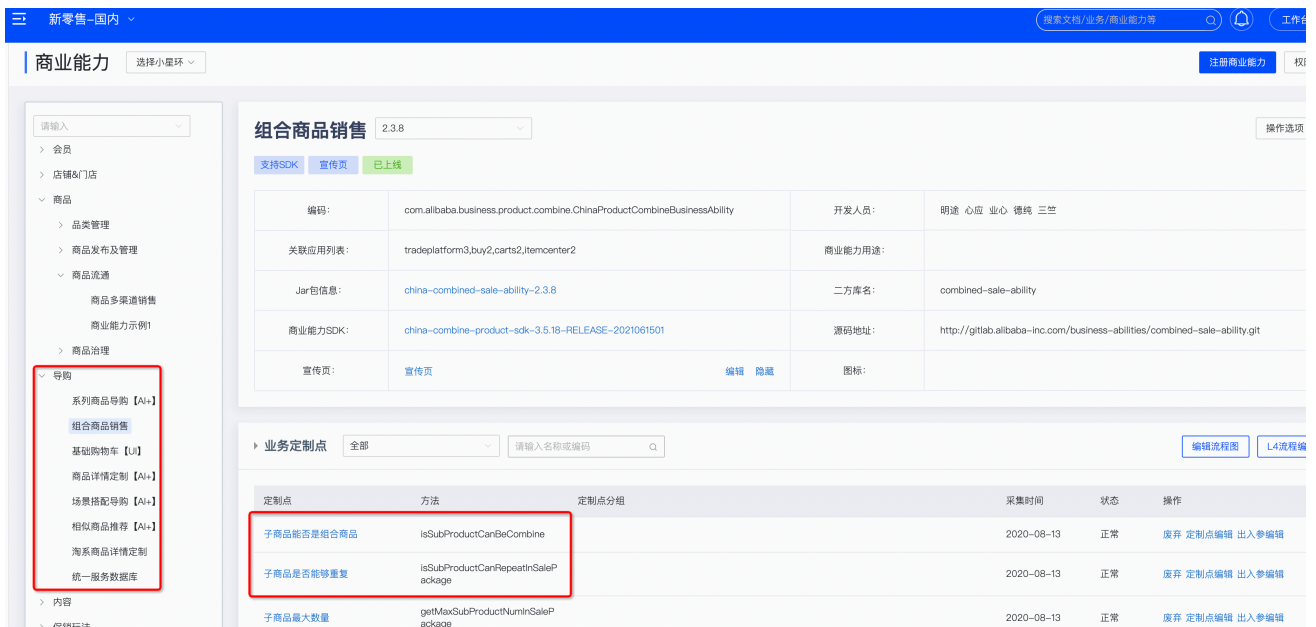
1. US服务的能力是否直接等于页面的功能集合
2. US的流程(Process、Module)是否等于页面的模块顺序
3. US是否具有业务流程
4. US的业务中心是什么

业务探索

商业能力

什么是商业能力

[什么是“商业能力” 星环概念汇总](#)



商业能力

为了达到某个独特的商业目的而提供的业务活动、系统能力和衡量标准的集合。特征如下：

- 商业能力一定是为了达到某个商业目的；
- 商业能力应当有自身的业务流程，通过多角色协作的业务活动达成业务目标；

商业能力跟域能力之间的区别，在于域能力是面向技术的语言，而商业能力是面向业务的语言。

商业能力扩展点

商业能力对外提供的可定制的方法级别的可扩展。

US的商业能力是什么

<https://us-conf.amap.com/#/home>

根据US的运营和扩展配置内容总结可知，US的商业能力扩展基本围绕用户和场景来进行页面视图和数据加工配置

首先：US并不具有具象的商业能力（导购、库存、营销等）和数据模型

因此：**US的商业能力是在具体的信息业务的商业能力之上的面向端（用户操作场景）的聚合能力**

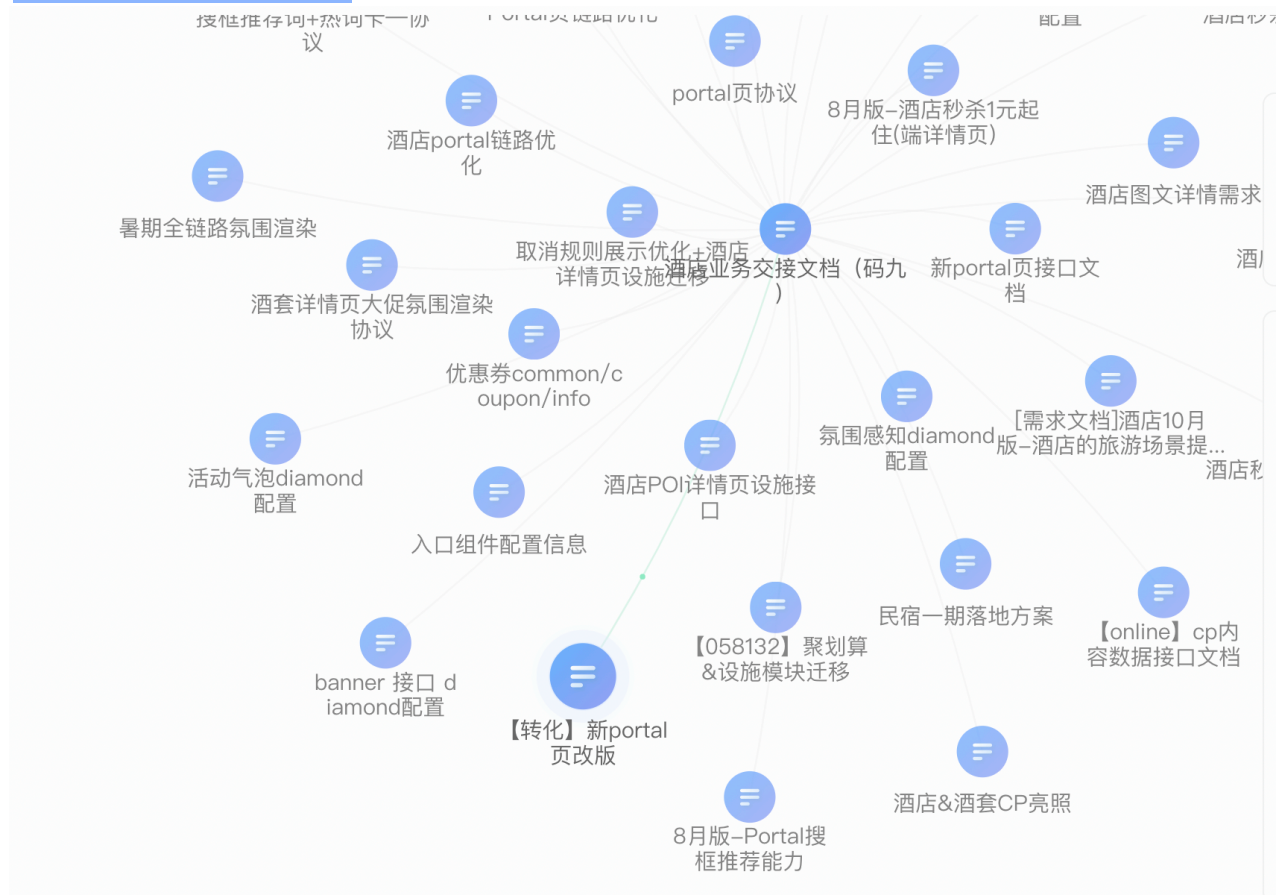
解释：US服务直接面向于端；而端是作为产品意图的直观呈现来承接用户的不同操作场景的存在

- us调用下游的商品、导购等服务能力来进行数据的召回
- us针对下游数据的召回提供针对用户、场景、页面级别的规则定制（能力扩展）

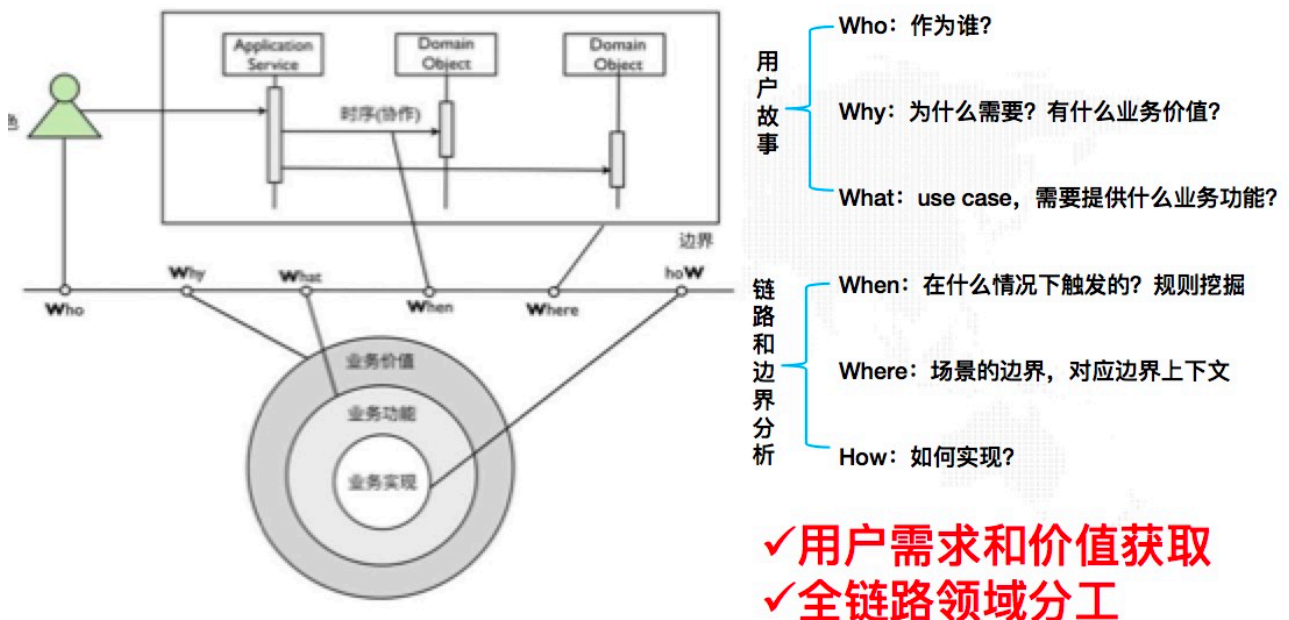
变与不变

产品方法论

【转化】新portal页改版



5W1H

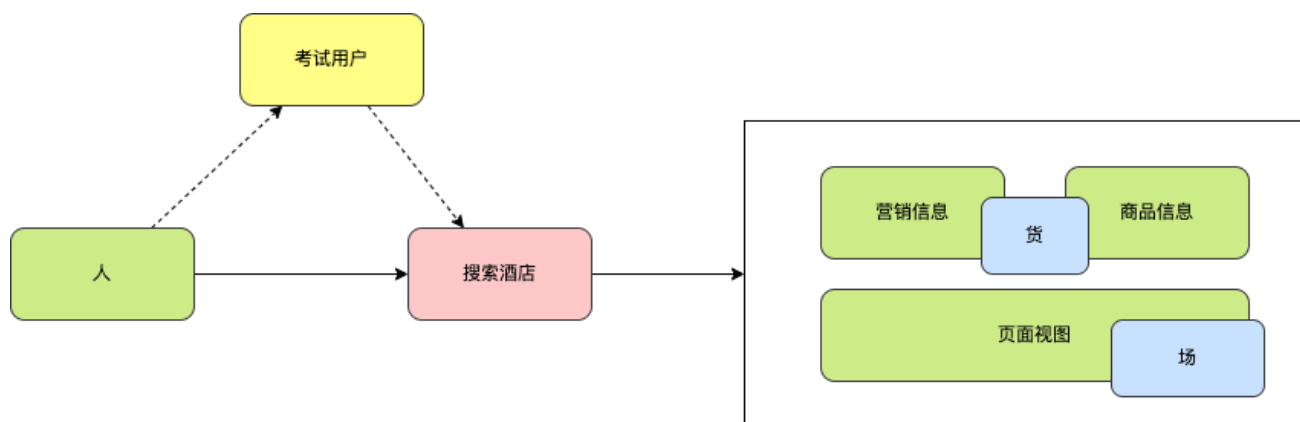


产品分析理论均是围绕用户和场景展开分析

将上面的产品方法论用近似的领域理论进行拆解分析

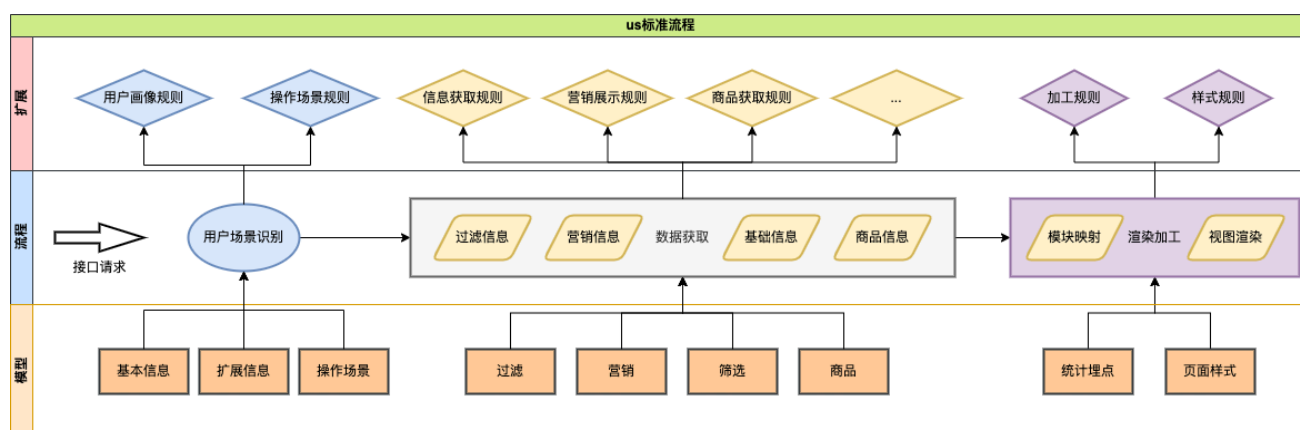
四色法

1. 时间 (Moment-Interval)：用户在端上的操作 (场景)
2. 人货场 (Party-Place-Thing Archetype)：人+呈现信息+呈现页面
3. 角色 (Role)：用户 (下沉、高价、商旅、考试等)
4. 描述 (Description)：略



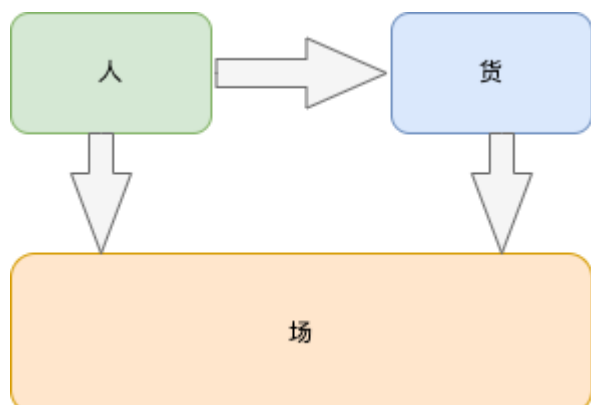
标准流程

由上可知，US所对接的业务基本都是围绕用户和其操作场景展开；可以总结出如下固定流程：



静态 (必须有)

- 流程：人->货->场
- 模型：用户基本信息、基于场景的返回数据
- 模块：基于用户场景的模块



- 动态（可没有）
 - 规则：画像、场景、营销、盘货、视图等
 - 模型：用户扩展信息、基于用户的返回数据
 - 模块：基于用户场景和返回数据的模块

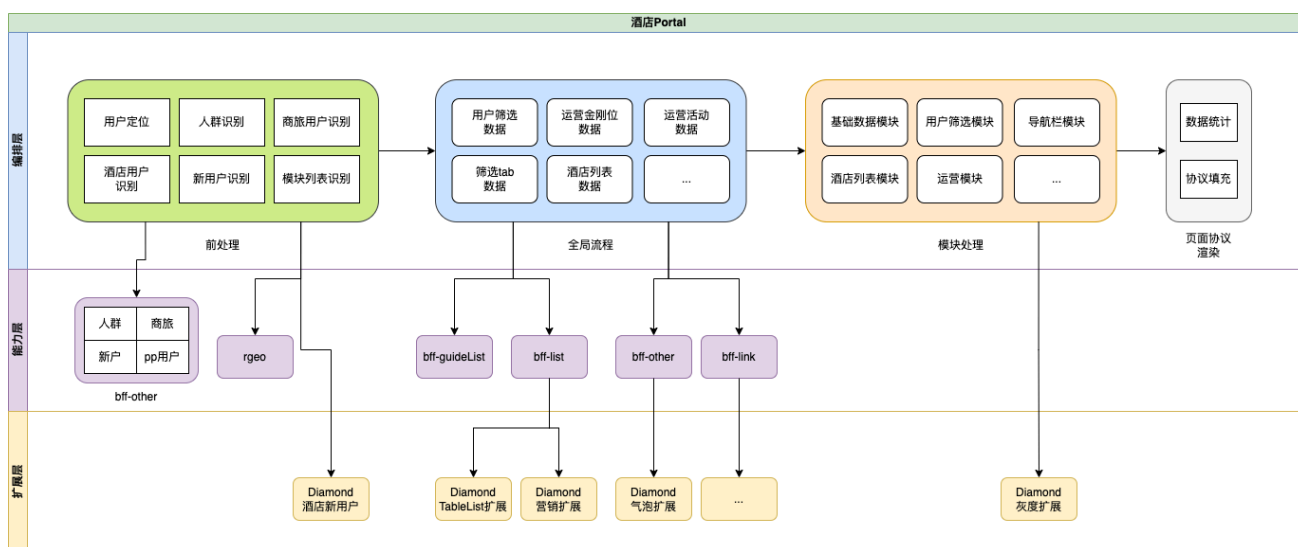
方案设计

首先分析实现现状，以酒店portal页为例

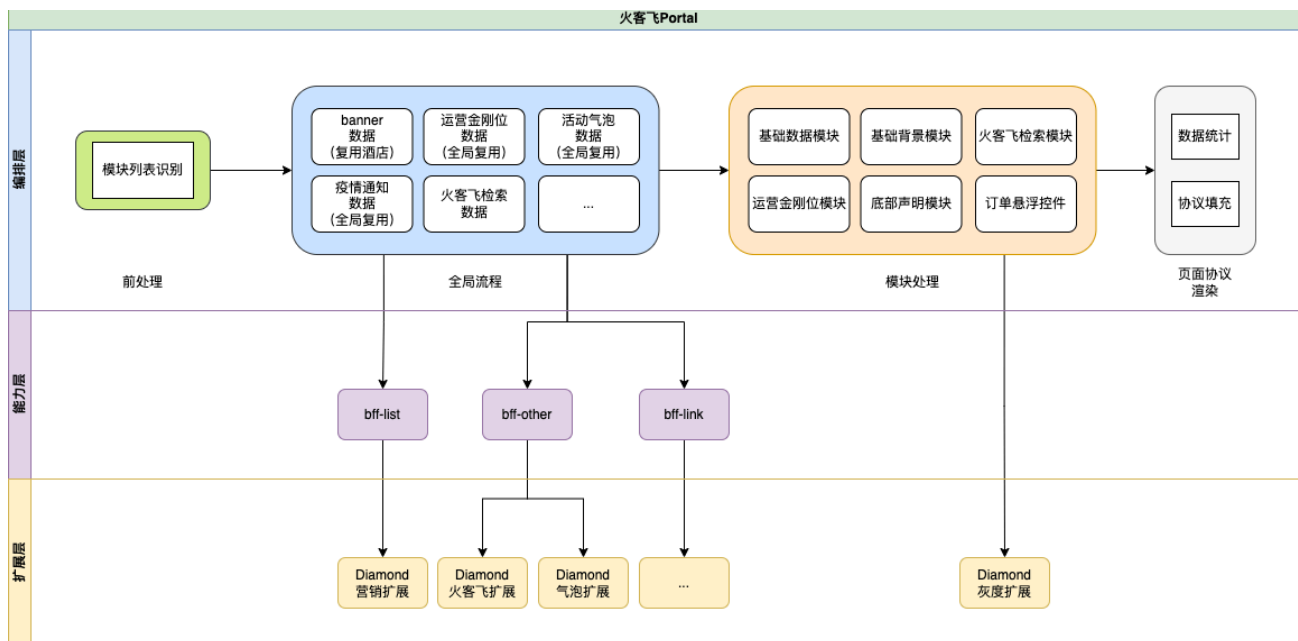
现有方案

现有实现大致分为2类，分别是不呈现数据与用户画像无关的页面和呈现数据与用户画像相关的页面：

- 酒店portal页，与用户画像有关的场景
 - 前处理会识别用户画像，加工参数
 - global基于用户和场景取数
 - module流程基于界面控件拆装global的取数结果



- 火客飞portal页，与用户画像无关的场景
 - 前处理无识别用户画像，只加工参数
 - global根据配置取数（运营类）
 - module流程基于界面控件拆装global的取数结果



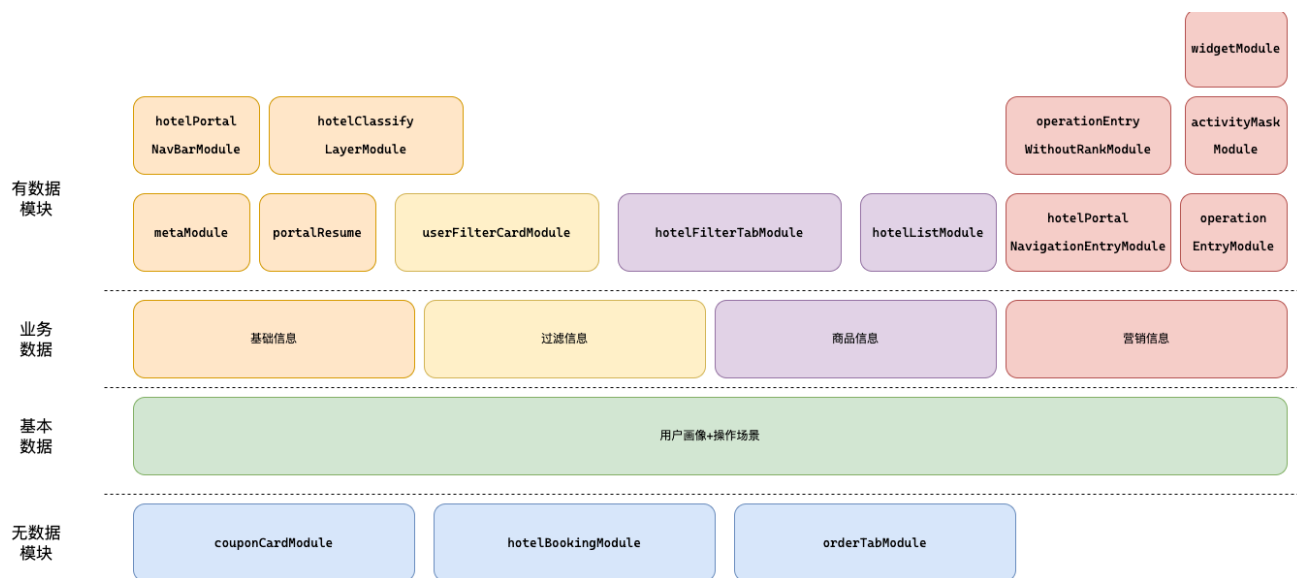
咨询过产品，结论是产品的页面基本都是围绕用户画像和操作场景来呈现，目前没有的未来会逐渐迭代补齐。

初衷

1. 架构分层
2. 能力复用
3. 扩展配置

现状

- 目前在US的日常开发中改动最频繁的在**render、calculator等VO层模块。 **
- 用户画像和行为场景的数据模型太薄，导致改用户场景需求经常需要改接口协议，扩大影响范围，降低效率
 - 增加子process处理新属性
 - 增加三方能力的调用fetcher（下游服务）
 - 修改受影响的接口协议（下游服务）
 - 修改关联的扩展逻辑（下游服务）
- 目前在已归类的服务中开始尝试将下游数据模型整合统一，屏蔽上游编排层对异构数据的处理
 - <https://yuque.antfin-inc.com/amap-search/gp80z4/ufhlx1vwde769vw1>
- 大多数接口流程的数据集中在了global流程，module流程基本只做数据的加工渲染逻辑
 - global适用于全局调用，所以往往会将多模块复用的数据获取放在global做
 - 实际中module和数据能力层并不是完全对应且一成不变，所以几乎所有数据的获取最终都在global
 - module层退化成了数据加工和渲染层的单一VO层
 - 这在us的实际业务中是合理的，因为业务及数据的划分并不等于页面模块



- 服务的划分选择了水平划分的方式，而产品输入一般以垂直业务为单位，所以每次迭代周期改动范围都比较大
 - 一个能力服务对接各垂类场景，每个垂类需求必改
 - 两个垂类需求同时输入就会导致同服务发布撞车

思考

- 页面及模块业务目标是否固定（例如：输入模块会不会加入导购属性）
 - 不固定，例如用户筛选可能加入营销控件
 - 模块不适合固定划分到对应流程中去（模块不等于业务流程）
- 业务数据是否可以固定下来
 - 可以，按类型可固定为基础信息、营销信息、商品信息等
 - 业务数据在下游可被加工映射到多个模块中返回端进行呈现
- 水平划分和垂直划分的差别
 - 水平：下游访问收口，垂类需求的服务改动范围较大
 - 垂直：业务聚类，下游能力的调用分散在业务流，较难复用

优化方案

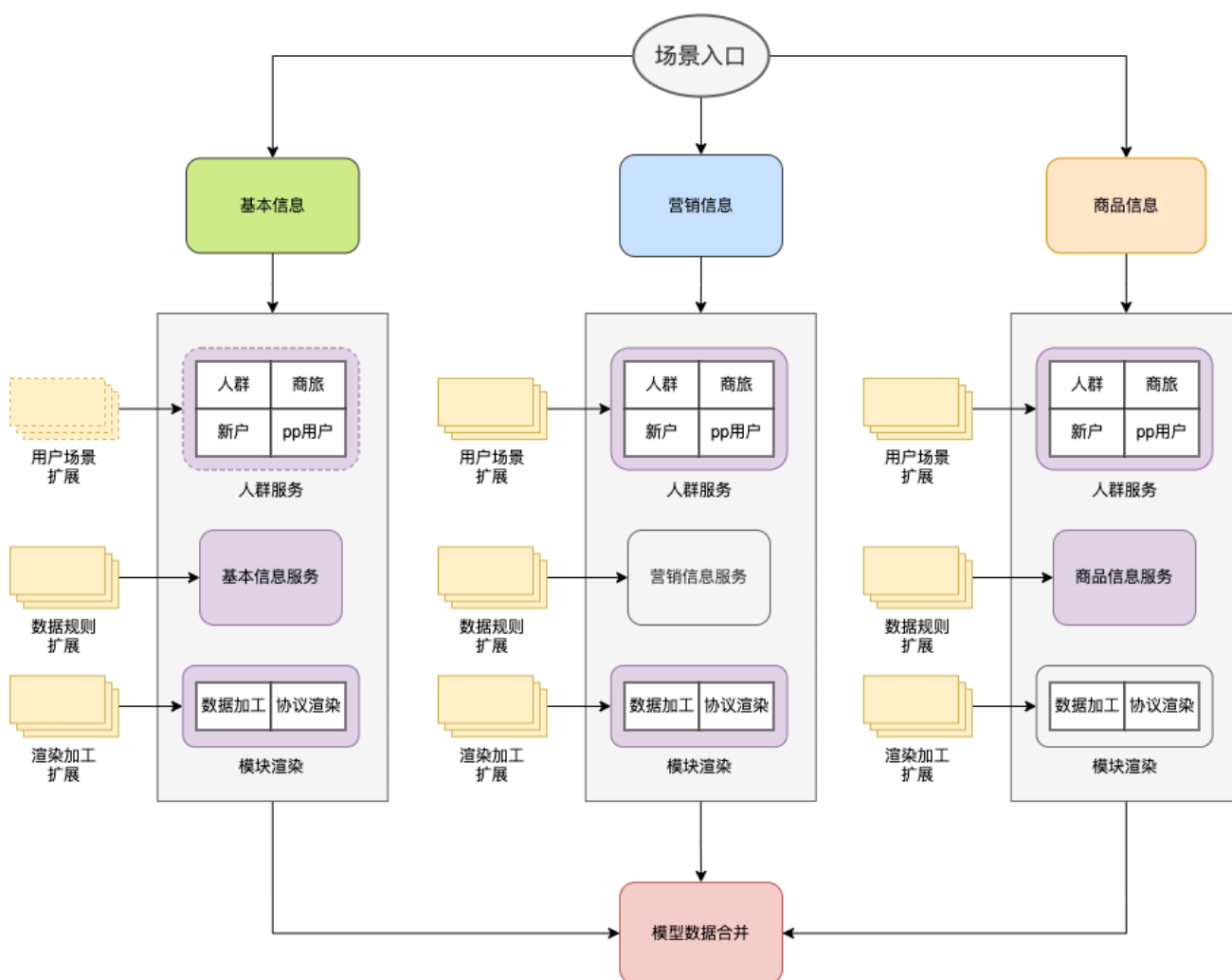
总体思路

- 核心：不同行业标准化
- 人
 - 用户场景沉淀为统一的能力
 - 提供通用的能力扩展（用户信息、顾客画像、行业场景）
- 货
 - 数据场景按照现有业态进行梳理，围绕人归纳为可复用的最小能力，例如
 - 基本信息

- 营销信息
- 商品信息
- 提供基于用户和场景的分级扩展
 - 一级扩展：模块列表动态执行（长期目标）
 - 二级扩展：模块中的数据处理规则
- 场
 - 页面场按照业态进行梳理，归纳为可复用页面模板
 - 提供基于人和货的，场的渲染加工扩展
 - 场中的视图渲染逻辑可抽象为设计平台（BFF-VO）

垂直划分方案

基于业务归纳的垂直划分（人群识别在子流程条件执行）



优点

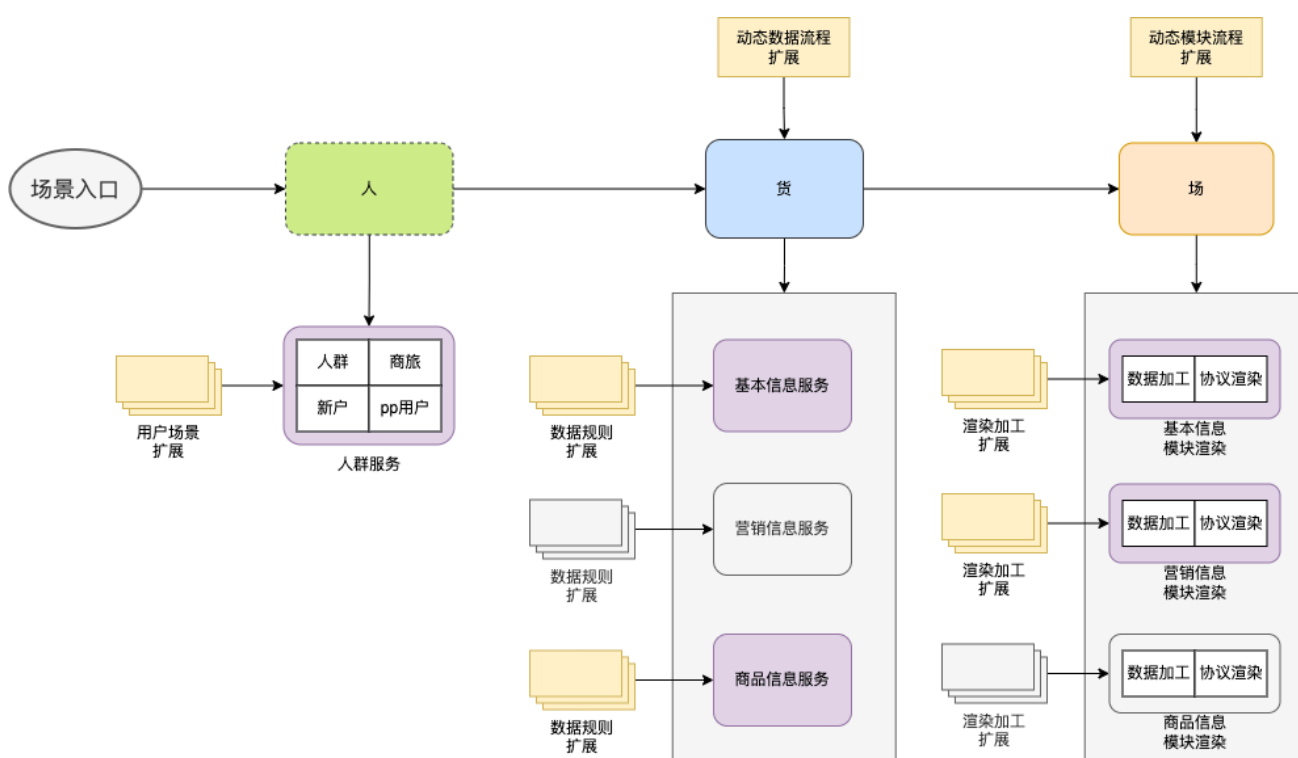
- 模块间的隔离性较好
- 整体过程可垂直并发执行
- 现有MIF和GBF均能较好支持

缺点

- 模块的动态执行策略判断时机比较靠后
 - 例如：营销信息的模块显示列表需要等前置的人群、信息服务执行结束后才能根据策略得出
- 如果存在跨业务取数的模块，最后需要一个总的合并节点来将不同业务中的同模块数据聚合
- 如出现不同业务的取数模块间依赖（货补依赖营销和货品召回），只能将其合并或在最后的数据合并节点处理

水平划分方案

基于标准流程的水平划分（人群识别在主流程条件执行）



优点

- 比较接近现有服务流程，改造难度较小
- 人群识别在整个过程只需一次，不用每个子流程单独访问
- 货和场的子流程节点可在总流程节点处配置扩展策略，根据前置结果动态执行
- 子流程的节点可并发执行，具有一定的隔离性
- 场的视图渲染逻辑在流程中比较集中，后期可整体迁移到BFF-VO或自建可视化平台，实现解耦

缺点

- 整体流程存在串行过程，对上游存在一定依赖
- 适合GBF，现有MIF不太合适

研发场景横向对比

场景	原有方案	垂直划分 (子流程在编排服务中)	垂直划分 (子流程在业务服务中)	水平划分
用户识别策略修改, 同时影响取数策略	+ 修改识别服务 + 修改编排服务的传参 + 改动协议参数 + 改动协议影响的服务 + 改动服务 1 + 1 + N	+ 修改识别服务 + 修改协议参数 + 改动协议影响流程 + 改动服务 1 + N	+ 修改识别服务 + 修改协议参数 + 改动协议影响流程的业务服务实现 + 改动服务 1 + N	+ 修改识别服务 + 修改协议参数 + 改动协议影响流程 + 改动服务 1 + N
用户识别策略修改, 同时影响模块显示	+ 修改识别服务 + 修改编排服务的流程 + 修改模块执行条件 + 改动服务 1 + 1	+ 修改识别服务 + 修改编排服务的流程执行条件 + 改动服务 1 + 1	+ 修改识别服务 + 修改业务服务的流程执行条件 + 改动服务 1 + N	+ 修改识别服务 + 修改编排服务的流程执行条件扩展 + 改动服务 1 + 改动扩展 1
数据获取策略修改, 同时影响显示逻辑	+ 修改对应数据服务 + 修改编排服务的上下文储存字段 + 修改编排服务 module和 render的加工显示逻辑 + 改动服务 1 + 1	+ 修改对应数据服务 + 修改编排服务的上下文模型 + 修改渲染流程的加工逻辑规则 + 改动服务 1 + 1	+ 修改对应数据服务 + 修改业务服务的渲染流程的加工逻辑规则 + 改动服务 1	+ 修改对应数据服务 + 修改编排服务的上下文模型 + 修改渲染流程的加工逻辑规则 + 改动服务 1 + 1

数据获取策略修改，同时影响模块显示	+ 修改对应数据服务 + 修改编排服务的上下文储存字段 + 修改模块执行条件 + 改动服务 1 + 1	+ 修改对应数据服务 + 修改编排服务的流程执行条件 + 改动服务 1 + 1	+ 修改对应数据服务 + 修改业务服务的流程执行条件 + 改动服务 1	+ 修改对应数据服务 + 修改编排服务的流程执行条件扩展 + 改动服务 1 + 改动扩展 1
修改基于上游数据的模块渲染逻辑	改动服务 1	改动扩展 1	+ 改动扩展 1	改动扩展 1
模块取数和业务功能的变更（显示模块变营销模块、商品模块增加营销数据）	改动服务 1	改动扩展 1	+ 改动服务 1 + 改动扩展 1	改动扩展 1

结论：

1. 垂直划分和水平划分的改动范围差异不大，均优于现有方案
 2. 垂直方案在业务取数环节变更时改动成本更低
 3. 水平方案在用户场景识别环节变更时改动成本更低
 4. 水平方案在业务取数子流程和模型渲染子流程能提供统一的动态规则配置
 5. 考虑US的商业能力定位是围绕用户来进行信息聚合，水平划分的方案从长远看更合适
- 选择水平划分的架构方案。

实施规划

1. 梳理us的业务服务根据现有流程归纳出来
2. 划分用户识别、业务取数、模块渲染三大流程边界
3. 按方案规划新的服务和流程
4. 抽取三类流程的规则扩展（二级扩展）
5. 将业务取数流程和模块渲染流程改造为规则扩展（一级扩展），动态执行（未来）
6. 模块视图渲染协议化、可视化

改造示例（酒店、火客飞Portal）：<https://code.alibaba-inc.com/gaode.arch/gbf-us-sample>

思考总结

- us是为了输出商业能力
 - 服务编排
 - 业务扩展
- 货（业务服务）的全集和标准流程需根据us组开发的梳理结果来定
- GBF比MIF的适用点
 - 代码级扩展的良好支持
 - 业务身份的识别、路由
 - 流程节点分布式调用的支持
 - 分层定义和规范
 - 流程编排（嵌套子流程）